S1. Tarea. Implementación de la estructura completa de una base de datos (DDL).

Por:

Johan Darío Martínez Echeverri

Jonathan Montoya Machado

Programa:

Tecnología en Desarrollo de Software

G100 (SD1006) Bases de Datos I

Oralia Cortes Grajales

Institución Universitaria Pascual Bravo

Medellín, Colombia

2026-2

1. **Elección y Justificación del SGBD:**
   Para el desarrollo de la Red Social Estudiantil Pascualina se realizó una evaluación comparativa entre tres de los motores de bases de datos más representativos del mercado: PostgreSQL, MySQL y Microsoft SQL Server.

   1.1. Cuadro Comparativo de SGBD:

Criterio / Característica	**PostgreSQL**	**MySQL**	**Microsoft SQL Server**
Tipo de Código / Licencia	Código Abierto (Open Source) (Licencia PostgreSQL - Estilo MIT/BSD)	Doble Licencia (GPL v2 / Licencia Comercial de Oracle)	Código Cerrado / Propietario (Licenciamiento por núcleos o CALs)
Costo de Licenciamiento	$0 USD (Completamente gratuito, sin restricciones de hardware o uso comercial/académico) .	Gratuito en Community Edition. Las versiones avanzadas y de soporte requieren pago a Oracle.	Alto costo. Existe versión Express gratuita, pero restringida a 1 GB de RAM y 10 GB por base de datos.
Gobernanza y Comunidad	Desarrollo independiente impulsado por una comunidad global sin fines de lucro. Sin monopolio corporativo.	Propiedad de Oracle Corporation. Las decisiones de desarrollo están condicionadas por intereses comerciales.	Propiedad exclusiva de Microsoft Corporation. Dependencia directa del fabricante (Vendor Lock-in).

Criterio / Característica	PostgreSQL	MySQL	Microsoft SQL Server
Cumplimiento ACID e Integridad	Estricto cumplimiento ACID por defecto. Excelente control de concurrencia mediante MVCC.	Cumple ACID en el motor InnoDB, pero con conversiones implícitas de datos menos rigurosas.	Estricto cumplimiento ACID, optimizado para entornos empresariales de Microsoft.
Tipos de Datos y Funcionalidades	Soporte superior para JSONB (semiestructurados), arreglos, tipos numéricos exactos y búsquedas de texto (Full-Text).	Soporte para JSON, pero con capacidades de indexación más limitadas sobre documentos.	Excelente integración con .NET y herramientas BI, pero con licencias costosas para análisis avanzado.
Herramienta Gráfica	pgAdmin 4: Interfaz moderna, multiplataforma, potente para edición DDL/DML y monitoreo.	MySQL Workbench: Herramienta de escritorio con modelado ER, aunque consume más recursos de sistema.	SQL Server Management Studio (SSMS): Muy completa, pero limitada principalmente al entorno Windows.

1.2. Elección de PostgreSQL

Las Ventajas del Código Abierto:

A diferencia de alternativas propietarias como SQL Server o de licenciamiento dual como MySQL:

- Sin costos de licenciamiento: Permite desplegar la base de datos en servidores de la institución educativa o en la nube sin incurrir en costos por núcleos de

CPU o límites de almacenamiento (ventaja clave frente a las limitaciones de SQL Server Express).

- Ausencia de Vendor Lock-in (Dependencia de Proveedor): PostgreSQL no pertenece a ninguna corporación privada. Esto garantiza que el sistema no quedará sujeto a cambios intempestivos en las políticas de cobro o licencias (un riesgo presente en MySQL tras la adquisición por parte de Oracle).
- Transparencia y Seguridad: El código fuente es auditable por la comunidad mundial, lo que permite la corrección rápida de vulnerabilidades de seguridad.

Robustez Técnica para Redes Sociales:

- Una red social universitaria requiere alta concurrencia y consultas complejas de relaciones (amigos de amigos, publicaciones por grupos, hilos de comentarios):
- Control de Concurrencia Multiversión (MVCC): Permite que múltiples estudiantes lean publicaciones mientras otros escriben comentarios o reaccionan, sin bloquear las tablas ni ralentizar el sistema.
- Flexibilidad de Datos: Si en el futuro la red social necesita almacenar configuraciones personalizadas de usuario o metadatos de publicaciones, el tipo de dato nativo JSONB de PostgreSQL permite mezclar el paradigma relacional con el no-relacional (NoSQL) de forma eficiente.

Integración con pgAdmin 4:

La gestión mediante pgAdmin 4 proporciona un entorno gráfico intuitivo para administrar las tablas, ejecutar el script DDL, inspeccionar las relaciones clave primaria/foránea y verificar visualmente las restricciones de integridad antes de realizar los envíos al repositorio de Git.

2. **Transformación del Modelo Lógico al Físico.**

Paso 1: Mapeo de Entidades Lógicas a Tablas Físicas

Cada entidad definida en el diagrama y en el documento de requerimientos fue traducida en una tabla física dentro del motor PostgreSQL 17:

- Entidades Principales o Fuertes: Se convirtieron en tablas independientes con identidad propia (estudiante, habilidad, interes, grupo, evento, publicacion).
- Entidades Débiles o Dependientes: Se convirtieron en tablas que extienden información o dependen directamente de una entidad fuerte, como perfil (relación 1:1 con estudiante) y comentario (dependiente de publicacion y estudiante).

- Relaciones de Muchos a Muchos (N:M): Se transformaron en 7 tablas asociativas o de intersección física (estudiante_habilidad, estudiante_interes, miembro_grupo, asistente_evento, reaccion, seguir y mentoria).

Paso 2: Conversión de Atributos a Tipos de Datos Físicos de PostgreSQL 17

Durante la transformación física, se asignaron tipos de datos nativos optimizados para cada campo:

- Claves Primarias Simples: Los atributos de identificación pasaron del tipo lógico INT a SERIAL en PostgreSQL (id_estudiante, id_perfil, id_habilidad, id_interes, id_grupo, id_evento, id_publicacion, id_comentario). El tipo SERIAL genera secuencias numéricas automáticas de 4 bytes, optimizando la indexación mediante B-Tree y la velocidad de los cruces (JOINs).
- Cadenas de Texto Limitadas: Los atributos de texto se tradujeron a VARCHAR(N), ajustando la longitud límite según la regla de negocio: VARCHAR(50) para nombres y apellidos, VARCHAR(100) para correos y temas, VARCHAR(255) para contraseñas cifradas (hash) y rutas de imágenes (foto_perfil), y VARCHAR(500) o VARCHAR(1000) para descripciones largas y publicaciones.
- Marcas Temporales: Todos los campos de fechas de registro, actualización e interacciones se definieron como TIMESTAMP con el valor por defecto DEFAULT CURRENT_TIMESTAMP para asegurar auditoría exacta del sistema.

Paso 3: Aplicación Formal de la Normalización en el Modelo Físico

El modelo físico refleja la aplicación de las tres primeras formas normales para eliminar redundancias y anomalías:

- Primera Forma Normal (1FN): Atributos multivaluados como las habilidades, intereses o grupos de un estudiante no se guardan concatenados en una sola celda. Fueron descompuestos en las tablas independientes habilidad, interes, estudiante_habilidad, estudiante_interes y miembro_grupo, garantizando atomicidad de datos.
- Segunda Forma Normal (2FN): En las tablas puente con clave primarias compuestas (estudiante_habilidad, estudiante_interes, miembro_grupo, asistente_evento, reaccion, seguir, mentoria), todos los atributos no clave dependientes (como nivel, rol o estado) dependen de la totalidad de la clave

primaria compuesta y no de una parte de ella. Los datos descriptivos del estudiante o de la habilidad permanecen en sus respectivas tablas principales.

- Tercera Forma Normal (3FN): Se eliminaron las dependencias transitivas en el esquema físico. Por ejemplo, no se almacena el nombre del estudiante dentro de publicacion o comentario, ni el nombre del grupo dentro de miembro_grupo. Toda la información suplementaria se obtiene en tiempo de ejecución mediante relaciones de clave foránea (FOREIGN KEY).

Paso 4: Implementación Física de las Relaciones e Integridad Referencial

Las cardinalidades del modelo lógico se tradujeron físicamente de la siguiente manera:

- Relación 1:1 (ESTUDIANTE – PERFIL): Se agregó la clave foránea id_estudiante en la tabla perfil con una restricción de unicidad (UNIQUE), garantizando que a cada estudiante le corresponda un solo perfil.
- Relaciones 1:N (ESTUDIANTE – PUBLICACION, GRUPO – EVENTO, PUBLICACION – COMENTARIO, etc.): La clave primaria de la entidad padre ("1") se migró como clave foránea en la tabla hija ("N").
- Relaciones N:M con Claves Primarias Compuestas: Se implementaron mediante tablas de intersección física cuyos campos de enlace conforman una clave primaria compuesta combinada PRIMARY KEY (col1, col2).
- Relaciones Autorreferenciales (N:M): Las tablas seguir (red de contactos) y mentoria conectan la tabla estudiante consigo misma. Físicamente se crearon dos claves foráneas apuntando a estudiante(id_estudiante): id_seguidor/id_seguido en seguir e id_mentor/id_aprendiz en mentoria.

Paso 5: Restricciones de Dominio y Reglas de Integridad Integradas (ON DELETE / CHECK)

En el script DDL ejecutable se incorporaron reglas de negocio físicas:

- Acción Referencial ON DELETE CASCADE: Aplicada en tablas dependientes como perfil, publicación, comentario, reacción, miembro_grupo, asistente_evento, estudiante_habilidad y estudiante_interes. Si se elimina un estudiante o una publicación, se limpian automáticamente todas sus interacciones secundarias asociadas sin dejar registros huérfanos.
- Acción Referencial ON DELETE RESTRICT: Implementada en la relación grupo.creador_id → estudiante.id_estudiante. Impide la eliminación accidental

de la cuenta de un estudiante si es propietario/creador de un grupo activo, protegiendo la existencia del grupo hasta que la propiedad sea reasignada.

- Restricciones CHECK de Negocio:
    - chk_no_auto_seguimiento: Restringe a nivel de motor SQL que un estudiante no pueda seguirse a sí mismo (CHECK (id_seguidor < >id_seguido)).
    - chk_no_auto_mentoria: Garantiza que un estudiante no pueda ser mentor de sí mismo (CHECK (id_mentor < > id_aprendiz)).

3. Descripción de cada tabla implementada

1. Tabla estudiante

Campos:

- id_estudiante SERIAL (PK)
- nombre VARCHAR(50) NOT NULL
- apellido VARCHAR(50) NOT NULL
- correo VARCHAR(100) NOT NULL UNIQUE
- password VARCHAR(255) NOT NULL
- fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
- ultima_actividad TIMESTAMP
- estado VARCHAR(20) DEFAULT 'Activo' NOT NULL

Clave Primaria (PK): id_estudiante.

Justificación de tipos de datos clave:

- id_estudiante es SERIAL (equivalente a INT + secuencia). Elegí INT y no BIGINT porque una red social estudiantil nunca va a superar los 2.147.483.647 registros; usar BIGINT desperdiciaría 4 bytes por fila sin necesidad. SERIAL me da el autoincremento automático y evita tener que generar el ID desde la aplicación.
- nombre, apellido y correo son VARCHAR y no TEXT porque son cadenas de longitud predecible y acotada. VARCHAR(n) permite a PostgreSQL optimizar el almacenamiento y validar la longitud máxima; TEXT no impone límite y no tendría sentido para un nombre.
- password es VARCHAR(255) y no CHAR ni TEXT porque ahí se guarda el hash de la contraseña (por ejemplo bcrypt o argon2), que ocupa un tamaño

variable pero acotado (típicamente 60–100 caracteres). 255 da margen suficiente y evita el relleno con espacios que haría CHAR.

- fecha_registro y ultima_actividad son TIMESTAMP y no DATE porque necesito precisión de hora, minuto y segundo (saber cuándo se registró y cuándo fue su última conexión). DATE solo guardaría el día.
- estado es VARCHAR(20) y no un ENUM ni un BOOLEAN porque quiero permitir valores como 'Activo', 'Inactivo', 'Suspendido' sin tener que alterar el tipo cada vez que se agregue un estado nuevo.
- correo lleva UNIQUE porque no puede haber dos estudiantes con el mismo correo; es un identificador natural dentro del sistema.

Justificación de la PK: Usé un ID numérico autoincremental porque es la forma más ligera y rápida de identificar a un usuario y relacionarlo con todas sus tablas hijas. Un INT de 4 bytes es más eficiente como clave foránea que un VARCHAR(100) de correo, y además el correo puede cambiar, mientras que el ID nunca.

2. Tabla perfil

Campos:

- id_perfil SERIAL (PK)
- id_estudiante INT NOT NULL UNIQUE (FK → estudiante)
- area_estudio VARCHAR(80)
- biografia VARCHAR(500)
- programa VARCHAR(80) NOT NULL
- semestre INT NOT NULL
- ciudad VARCHAR(50)
- foto_perfil VARCHAR(255)
- fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): id_perfil.

Justificación de tipos de datos clave:

- id_perfil es SERIAL por la misma razón que en estudiante: autoincremento simple y eficiente.
- id_estudiante es INT (no BIGINT) porque debe coincidir exactamente con el tipo de la PK de estudiante; si fuera BIGINT, la FK ocuparía el doble y no habría beneficio.

- biografia es VARCHAR(500) y no TEXT porque una biografía de perfil tiene un límite razonable; si fuera TEXT podría llegar a almacenar megabytes sin control.
- semestre es INT y no SMALLINT porque, aunque un semestre cabría en SMALLINT, INT es el tipo estándar para números enteros en PostgreSQL y evita conversiones implícitas.
- fecha_actualizacion es TIMESTAMP porque necesito saber la fecha y hora exacta de la última modificación del perfil.
- id_estudiante lleva UNIQUE porque la relación es 1:1: un estudiante solo puede tener un perfil.

Justificación de la PK: Aunque es una relación 1:1 y podría haberse usado id_estudiante como PK, decidí darle su propia PK (id_perfil) para mantener el estándar del esquema (todas las tablas con PK propia) y porque a futuro podría querer permitir más de un perfil por estudiante. El UNIQUE en id_estudiante garantiza la unicidad actual.

3. Tabla habilidad

Campos:

- id_habilidad SERIAL (PK)
- nombre VARCHAR(50) NOT NULL UNIQUE
- descripcion VARCHAR(255)

Clave Primaria (PK): id_habilidad.

Justificación de tipos de datos clave:

- id_habilidad es SERIAL porque es un catálogo y necesito un identificador numérico ligero para referenciarlo desde estudiante_habilidad.
- nombre es VARCHAR(50) y no TEXT porque los nombres de habilidades son cortos y acotados ("Python", "Trabajo en equipo", etc.). UNIQUE evita duplicar la misma habilidad en el catálogo.
- descripcion es VARCHAR(255) porque es un texto breve explicativo; no necesito TEXT para una descripción de catálogo.

Justificación de la PK: Usar un número como PK hace que las búsquedas y los JOIN sean más rápidos que buscar por el nombre en texto. Además, el nombre podría cambiar (por ejemplo, corregir una tilde) sin afectar las relaciones.

4. Tabla estudiante_habilidad

Campos:

- id_estudiante INT NOT NULL (FK → estudiante)

- id_habilidad INT NOT NULL (FK → habilidad)
- nivel VARCHAR(20) NOT NULL
- fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_estudiante, id_habilidad).

Justificación de tipos de datos clave:

- Ambos IDs son INT porque deben coincidir con las PK de estudiante y habilidad respectivamente. No uso BIGINT porque no aporta nada y duplicaría el tamaño de la clave compuesta.
- nivel es VARCHAR(20) y no un ENUM porque quiero permitir valores como 'Básico', 'Intermedio', 'Avanzado' sin tener que modificar el tipo si agrego uno nuevo.
- fecha_agregado es TIMESTAMP porque necesito la fecha y hora exacta en que el estudiante registró esa habilidad.

Justificación de la PK: Es una clave compuesta por (id_estudiante, id_habilidad). Mi justificación es evitar datos duplicados: si uso esta combinación como PK, la base de datos no dejará que un mismo estudiante registre la misma habilidad dos veces. Además, la PK compuesta genera automáticamente un índice que acelera las búsquedas por esa combinación.

5. Tabla interes

Campos:

- id_interes SERIAL (PK)
- nombre VARCHAR(50) NOT NULL UNIQUE
- descripcion VARCHAR(255)

Clave Primaria (PK): id_interes.

Justificación de tipos de datos clave:

- Igual que en habilidad: SERIAL para el ID, VARCHAR(50) para el nombre (corto y único), VARCHAR(255) para la descripción breve.
- No uso TEXT porque los intereses son etiquetas cortas y una descripción de catálogo no necesita más de 255 caracteres.

Justificación de la PK: Al igual que las habilidades, es una tabla de catálogo para estandarizar los intereses. Un ID numérico es más eficiente como FK que el nombre en texto.

6. Tabla estudiante_interes

Campos:

- id_estudiante INT NOT NULL (FK → estudiante)
- id_interes INT NOT NULL (FK → interes)
- fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_estudiante, id_interes).

Justificación de tipos de datos clave:

- INT para ambos IDs porque coinciden con las PK referenciadas.
- fecha_agregado es TIMESTAMP para registrar el momento exacto en que se agregó el interés.

Justificación de la PK: Compuesta por (id_estudiante, id_interes). La elegí así para garantizar que un estudiante no pueda agregar el mismo interés varias veces a su perfil. También sirve como índice para búsquedas por estudiante o por interés.

7. Tabla grupo

Campos:

- id_grupo SERIAL (PK)
- creador_id INT NOT NULL (FK → estudiante)
- nombre VARCHAR(80) NOT NULL
- descripcion VARCHAR(500)
- tipo VARCHAR(30) NOT NULL
- fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
- estado VARCHAR(20) DEFAULT 'Activo' NOT NULL

Clave Primaria (PK): id_grupo.

Justificación de tipos de datos clave:

- id_grupo es SERIAL para tener un identificador numérico ligero que se referencia desde miembro_grupo y evento.
- creador_id es INT porque debe coincidir con la PK de estudiante.
- nombre es VARCHAR(80) y no TEXT porque los nombres de grupo son cortos pero pueden ser un poco más largos que un nombre de persona; 80 es suficiente.

- descripcion es VARCHAR(500) porque una descripción de grupo puede ser más extensa que una de catálogo, pero sigue siendo acotada.
- tipo y estado son VARCHAR(30) y VARCHAR(20) respectivamente, para permitir valores flexibles sin ENUM.
- fecha_creacion es TIMESTAMP para saber cuándo se creó el grupo.

Justificación de la PK: Identificador numérico estándar para poder enlazar fácilmente a los miembros y los eventos del grupo. Un INT es más eficiente como FK que un VARCHAR(80) con el nombre del grupo.

8. Tabla miembro_grupo

Campos:

- id_estudiante INT NOT NULL (FK → estudiante)
- id_grupo INT NOT NULL (FK → grupo)
- fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
- rol VARCHAR(30) DEFAULT 'Miembro' NOT NULL

Clave Primaria (PK): Compuesta por (id_estudiante, id_grupo).

Justificación de tipos de datos clave:

- INT para ambos IDs porque coinciden con las PK referenciadas.
- fecha_ingreso es TIMESTAMP para registrar el momento exacto de ingreso.
- rol es VARCHAR(30) para permitir valores como 'Miembro', 'Administrador', 'Moderador' sin ENUM.

Justificación de la PK: Compuesta por (id_estudiante, id_grupo). Me pareció la mejor opción para asegurar que una persona no aparezca unida al mismo grupo dos veces. Además, la PK compuesta crea un índice que acelera las consultas de "¿qué grupos tiene este estudiante?" y "¿quiénes son los miembros de este grupo?".

9. Tabla evento

Campos:

- id_evento SERIAL (PK)
- id_grupo INT NOT NULL (FK → grupo)
- nombre VARCHAR(100) NOT NULL
- descripcion VARCHAR(500)
- fecha_inicio TIMESTAMP NOT NULL

- fecha_fin TIMESTAMP NOT NULL
- lugar VARCHAR(150)
- modalidad VARCHAR(30) NOT NULL
- organizador VARCHAR(100) NOT NULL
- fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): id_evento.

Justificación de tipos de datos clave:

- id_evento es SERIAL para tener un ID numérico ligero referenciado desde asistente_evento.
- id_grupo es INT porque coincide con la PK de grupo.
- nombre es VARCHAR(100) porque los nombres de evento pueden ser más largos que los de grupo.
- fecha_inicio y fecha_fin son TIMESTAMP y no DATE porque un evento puede empezar y terminar el mismo día a distintas horas; necesito precisión horaria.
- lugar es VARCHAR(150) porque una dirección o nombre de lugar puede ser más largo.
- modalidad es VARCHAR(30) para permitir 'Presencial', 'Virtual', 'Híbrida'.
- organizador es VARCHAR(100) y no una FK a estudiante porque el organizador puede ser una persona externa, un departamento o una entidad; se guarda como texto libre.

Justificación de la PK: id_evento es el identificador numérico estándar que permite enlazar los asistentes al evento sin depender de combinaciones de campos.

10. Tabla asistente_evento

Campos:

- id_estudiante INT NOT NULL (FK → estudiante)
- id_evento INT NOT NULL (FK → evento)
- estado VARCHAR(20) DEFAULT 'Confirmado' NOT NULL
- fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_estudiante, id_evento).

Justificación de tipos de datos clave:

- INT para ambos IDs porque coinciden con las PK referenciadas.

- estado es VARCHAR(20) para permitir 'Confirmado', 'Pendiente', 'Cancelado'.
- fecha_registro es TIMESTAMP para saber cuándo se registró la asistencia.

Justificación de la PK: Compuesta por (id_estudiante, id_evento). Decidí usarla para que la confirmación de asistencia de un estudiante a un evento sea única: un estudiante no puede registrarse dos veces al mismo evento.

11. Tabla publicacion

Campos:

- id_publicacion SERIAL (PK)
- id_estudiante INT NOT NULL (FK → estudiante)
- contenido VARCHAR(1000) NOT NULL
- tipo VARCHAR(30) NOT NULL
- fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): id_publicacion.

Justificación de tipos de datos clave:

- id_publicacion es SERIAL para tener un ID numérico ligero referenciado desde reaccion y comentario.
- id_estudiante es INT porque coincide con la PK de estudiante.
- contenido es VARCHAR(1000) y no TEXT porque, aunque es un texto más largo que los anteriores, sigue siendo acotado (una publicación de red social no debería ser ilimitada). VARCHAR(1000) impone un límite razonable y evita publicaciones gigantes.
- tipo es VARCHAR(30) para permitir 'Texto', 'Imagen', 'Enlace', etc.
- fecha_publicacion es TIMESTAMP para ordenar cronológicamente las publicaciones.

Justificación de la PK: id_publicacion es necesaria para que luego podamos enlazarle los comentarios y las reacciones a ese post en específico. Un INT autoincremental es más eficiente como FK que una combinación de id_estudiante + fecha_publicacion.

12. Tabla reaccion

Campos:

- id_estudiante INT NOT NULL (FK → estudiante)

- id_publicacion INT NOT NULL (FK → publicacion)
- tipo_reaccion VARCHAR(20) NOT NULL
- fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_estudiante, id_publicacion).

Justificación de tipos de datos clave:

- INT para ambos IDs porque coinciden con las PK referenciadas.
- tipo_reaccion es VARCHAR(20) para permitir 'Like', 'Love', 'Dislike', etc.
- fecha es TIMESTAMP para saber cuándo se reaccionó.

Justificación de la PK: Compuesta por (id_estudiante, id_publicacion). Esta fue una decisión importante: la PK compuesta asegura que un usuario solo pueda dejarle un "Like" (o una reacción) a una misma publicación. No puede reaccionar dos veces. Si quisiera permitir múltiples reacciones distintas del mismo usuario a la misma publicación, tendría que incluir tipo_reaccion en la PK, pero el diseño actual asume una sola reacción por usuario y publicación.

13. Tabla comentario

Campos:

- id_comentario SERIAL (PK)
- id_publicacion INT NOT NULL (FK → publicacion)
- id_estudiante INT NOT NULL (FK → estudiante)
- comentario VARCHAR(500) NOT NULL
- fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): id_comentario.

Justificación de tipos de datos clave:

- id_comentario es SERIAL porque aquí SÍ necesito un ID propio: un mismo estudiante puede comentar varias veces la misma publicación, así que no puedo usar una PK compuesta por (id_estudiante, id_publicacion).
- id_publicacion e id_estudiante son INT porque coinciden con las PK referenciadas.
- comentario es VARCHAR(500) y no TEXT porque un comentario es más corto que una publicación; 500 caracteres es un límite razonable.
- fecha_comentario es TIMESTAMP para ordenar los comentarios cronológicamente.

Justificación de la PK: Aquí SÍ usé un SERIAL autoincrementar porque, a diferencia de las reacciones, un mismo estudiante sí puede comentar varias veces en la misma publicación. Necesito un identificador único por comentario.

14. Tabla seguir

Campos:

- id_seguidor INT NOT NULL (FK → estudiante)
- id_seguido INT NOT NULL (FK → estudiante)
- fecha_seguimiento TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_seguidor, id_seguido).

Justificación de tipos de datos clave:

- Ambos son INT porque referencian a estudiante(id_estudiante).
- fecha_seguimiento es TIMESTAMP para saber cuándo empezó a seguir a esa persona.

Justificación de la PK: Compuesta por (id_seguidor, id_seguido). Evita que yo pueda seguir a la misma persona más de una vez. Además, la restricción CHECK (id_seguidor <> id_seguido) impide que un estudiante se siga a sí mismo.

15. Tabla mentoria

Campos:

- id_mentor INT NOT NULL (FK → estudiante)
- id_aprendiz INT NOT NULL (FK → estudiante)
- tema VARCHAR(100) NOT NULL
- estado VARCHAR(20) DEFAULT 'Solicitado' NOT NULL
- fecha_mentoria TIMESTAMP NOT NULL

Clave Primaria (PK): Compuesta por (id_mentor, id_aprendiz).

Justificación de tipos de datos clave:

- id_mentor e id_aprendiz son INT porque referencian a estudiante(id_estudiante).
- tema es VARCHAR(100) porque el tema de una mentoría puede ser una frase corta pero más descriptiva que un nombre.
- estado es VARCHAR(20) para permitir 'Solicitado', 'Aceptado', 'Rechazado', 'Completado'.

- fecha_mentoria es TIMESTAMP porque necesito fecha y hora de la sesión.

Justificación de la PK: Compuesta por (id_mentor, id_aprendiz). Evita duplicidad en la solicitud de mentoría entre el mismo par de estudiantes. La restricción CHECK (id_mentor <> id_aprendiz) impide que un estudiante se pida mentoría a sí mismo.

4. **Implementación de las relaciones entre tablas**

Nota sobre ON UPDATE: Las claves primarias numéricas generadas automáticamente con SERIAL no se modifican normalmente durante el funcionamiento de la aplicación. En el script no se especifica una acción ON UPDATE para las claves foráneas, por lo que PostgreSQL utiliza NO ACTION por defecto. Esto evita que una actualización de la clave referenciada se realice si deja relaciones inconsistentes. La misma regla se aplica a las claves foráneas que referencian claves primarias compuestas. no tiene sentido actualizar en cascada un ID que nunca se modifica.

1. Relación estudiante → perfil

- Tabla padre: estudiante
- Tabla hija: perfil
- Campo FK: perfil.id_estudiante → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: El perfil depende existencialmente del estudiante. Si un estudiante se retira y elimino su cuenta, su perfil debe borrarse automáticamente para no dejar basura en la base de datos. Además, como id_estudiante es UNIQUE en perfil, la relación es 1:1.

2. Relación estudiante → estudiante_habilidad

- Tabla padre: estudiante
- Tabla hija: estudiante_habilidad
- Campo FK: estudiante_habilidad.id_estudiante → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, quiero que el sistema limpie automáticamente todas las habilidades que tenía asignadas. No tiene sentido conservar registros de habilidades de un usuario que ya no existe.

3. Relación habilidad → estudiante_habilidad

- Tabla padre: habilidad
- Tabla hija: estudiante_habilidad
- Campo FK: estudiante_habilidad.id_habilidad → habilidad.id_habilidad
- ON DELETE: CASCADE
- Justificación: Si elimino una habilidad del catálogo (por ejemplo, porque ya no se usa), las asignaciones de esa habilidad a los estudiantes pierden sentido y deben borrarse en cascada.

4. Relación estudiante → estudiante_interes

- Tabla padre: estudiante
- Tabla hija: estudiante_interes
- Campo FK: estudiante_interes.id_estudiante → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Al eliminar un estudiante, sus intereses asociados ya no tienen razón de existir y se borran automáticamente.

5. Relación interes → estudiante_interes

- Tabla padre: interes
- Tabla hija: estudiante_interes
- Campo FK: estudiante_interes.id_interes → interes.id_interes
- ON DELETE: CASCADE
- Justificación: Si se elimina un interés del catálogo, las asignaciones de ese interés a los estudiantes deben desaparecer para no dejar referencias huérfanas.

6. Relación estudiante → grupo (creador)

- Tabla padre: estudiante
- Tabla hija: grupo
- Campo FK: grupo.creador_id → estudiante.id_estudiante
- ON DELETE: RESTRICT
- Justificación: Tomé esta decisión para proteger a los usuarios. Si el creador de un grupo se da de baja, no quiero usar CASCADE porque borraría el grupo

entero, expulsando a todos los demás miembros sin aviso. Al usar RESTRICT, el sistema arrojará un error si intento borrar a ese estudiante, lo que obligará a que primero se le transfiera el rol de creador a otro estudiante antes de poder borrar la cuenta. Es una excepción estratégica dentro del modelo.

7. Relación estudiante → miembro_grupo

- Tabla padre: estudiante
- Tabla hija: miembro_grupo
- Campo FK: miembro_grupo.id_estudiante → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, quiero que desaparezca automáticamente de todos los grupos a los que pertenecía. No tiene sentido dejar membresías de usuarios inexistentes.

8. Relación grupo → miembro_grupo

- Tabla padre: grupo
- Tabla hija: miembro_grupo
- Campo FK: miembro_grupo.id_grupo → grupo.id_grupo
- ON DELETE: CASCADE
- Justificación: Si elimino un grupo, todas sus membresías deben borrarse en cascada porque ya no existe el grupo al que pertenecían.

9. Relación grupo → evento

- Tabla padre: grupo
- Tabla hija: evento
- Campo FK: evento.id_grupo → grupo.id_grupo
- ON DELETE: CASCADE
- Justificación: Un evento le pertenece a un grupo. Si el grupo se elimina (por inactividad o porque lo cerraron), todos los eventos que estaban planeando ya no tienen sentido y deben desaparecer.

10. Relación estudiante → asistente_evento

- Tabla padre: estudiante

- Tabla hija: asistente_evento
- Campo FK: asistente_evento.id_estudiante → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, sus confirmaciones de asistencia a eventos deben borrarse automáticamente.

11. Relación evento → asistente_evento
    - Tabla padre: evento
    - Tabla hija: asistente_evento
    - Campo FK: asistente_evento.id_evento → evento.id_evento
    - ON DELETE: CASCADE
    - Justificación: Si se elimina un evento, las asistencias registradas a ese evento ya no tienen sentido y se borran en cascada.

12. Relación estudiante → publicacion
    - Tabla padre: estudiante
    - Tabla hija: publicacion
    - Campo FK: publicacion.id_estudiante → estudiante.id_estudiante
    - ON DELETE: CASCADE
    - Justificación: Todo el contenido (posts) le pertenece al autor. Si el autor se va de la red, su contenido se elimina.

13. Relación estudiante → reaccion
    - Tabla padre: estudiante
    - Tabla hija: reaccion
    - Campo FK: reaccion.id_estudiante → estudiante.id_estudiante
    - ON DELETE: CASCADE
    - Justificación: Si elimino a un estudiante, quiero que el sistema limpie automáticamente los "likes" que había dado.

14. Relación publicacion → reaccion
    - Tabla padre: publicacion

- Tabla hija: reaccion
- Campo FK: reaccion.id_publicacion → publicacion.id_publicacion
- ON DELETE: CASCADE
- Justificación: Si borro una publicación, no tiene sentido conservar sus reacciones; se borran en cascada.

15. Relación publicacion → comentario
    - Tabla padre: publicacion
    - Tabla hija: comentario
    - Campo FK: comentario.id_publicacion → publicacion.id_publicacion
    - ON DELETE: CASCADE
    - Justificación: Si borro un post, todos los comentarios de ese post se tienen que borrar para no quedar volando.

16. Relación estudiante → comentario
    - Tabla padre: estudiante
    - Tabla hija: comentario
    - Campo FK: comentario.id_estudiante → estudiante.id_estudiante
    - ON DELETE: CASCADE
    - Justificación: Si el autor se va de la red, sus comentarios se eliminan. Todo el contenido le pertenece al autor, igual que las publicaciones.

17. Relación autoreferencial estudiante → seguir (seguidor)
    - Tabla padre: estudiante
    - Tabla hija: seguir
    - Campo FK: seguir.id_seguidor → estudiante.id_estudiante
    - ON DELETE: CASCADE
    - Justificación: Si elimino a un estudiante, se borran sus registros de a quién seguía.

18. Relación autoreferencial estudiante → seguir (seguido)
    - Tabla padre: estudiante

- Tabla hija: seguir
- Campo FK: seguir.id_seguido → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, también se borran los registros de quiénes lo seguían a él, para no dejar referencias huérfanas.
- Restricción extra: CHECK (id_seguidor <> id_seguido) porque un estudiante no debería poder seguirse a sí mismo.

19. Relación autoreferencial estudiante → mentoria (mentor)

- Tabla padre: estudiante
- Tabla hija: mentoria
- Campo FK: mentoria.id_mentor → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, se borran las mentorías en las que participaba como mentor.

20. Relación autoreferencial estudiante → mentoria (aprendiz)

- Tabla padre: estudiante
- Tabla hija: mentoria
- Campo FK: mentoria.id_aprendiz → estudiante.id_estudiante
- ON DELETE: CASCADE
- Justificación: Si elimino a un estudiante, se borran las mentorías en las que participaba como aprendiz.
- Restricción extra: CHECK (id_mentor <> id_aprendiz) porque un estudiante no debería poder pedirse una mentoría a sí mismo.

Resumen de acciones referenciales

#	Tabla padre	Tabla hija	Campo FK	ON DELETE
1	estudiante	perfil	id_estudiante	CASCADE
2	estudiante	estudiante_habilidad	id_estudiante	CASCADE
3	habilidad	estudiante_habilidad	id_habilidad	CASCADE
4	estudiante	estudiante_interes	id_estudiante	CASCADE
5	interes	estudiante_interes	id_interes	CASCADE
6	estudiante	grupo	creador_id	RESTRICT
7	estudiante	miembro_grupo	id_estudiante	CASCADE

8	grupo	miembro_grupo	id_grupo	CASCADE
9	grupo	evento	id_grupo	CASCADE
10	estudiante	asistente_evento	id_estudiante	CASCADE
11	evento	asistente_evento	id_evento	CASCADE
12	estudiante	publicacion	id_estudiante	CASCADE
13	estudiante	reaccion	id_estudiante	CASCADE
14	publicacion	reaccion	id_publicacion	CASCADE
15	publicacion	comentario	id_publicacion	CASCADE
16	estudiante	comentario	id_estudiante	CASCADE
17	estudiante	seguir	id_seguidor	CASCADE

5. Explicación y Referencia del Script DDL:

- Limpieza del Esquema (CASCADE): Permite ejecutar el script en entornos de desarrollo sin errores por objetos preexistentes.
- Creación de tablas independientes: se crean primero las tablas estudiante, habilidad e interes. Estas entidades no dependen de ninguna otra tabla y generan las claves primarias (SERIAL) utilizadas en el resto del modelo.
- Creación de Entidades Dependientes: Se definen las tablas perfil, grupo, publicacion, seguir y mentoria, las cuales hacen referencia directa a la tabla estudiante mediante claves foráneas.
- Creación de entidades dependientes: se construyen las tablas evento, comentario, reaccion, miembro_grupo, asistente_evento, estudiante_habilidad y estudiante_interes.
- Validación de Restricciones Integradas: El script declara restricciones a nivel de columna y de tabla, incluyendo el uso de CHECK para prevenir auto-seguimiento (chk_no_auto_seguimiento) y auto-mentoría (chk_no_auto_mentoria), así como cláusulas UNIQUE para garantizar la integridad en correos y nombres de catálogo.

6. Conclusiones individuales:

JDME

Trabajar en el diseño físico de esta base de datos me dejó aprendizajes muy valiosos sobre la arquitectura de software. Entendí que la integridad referencial no es solo una regla teórica, sino la garantía de que la plataforma no quede con datos 'basura'. Por ejemplo, me pareció fundamental configurar las reglas ON DELETE CASCADE en tablas como comentario y reaccion, ya que, si se elimina una publicación, el motor limpia automáticamente todas sus interacciones sin dejar registros huérfanos.

Durante el desarrollo también enfrenté algunas dificultades. Lo más complejo fue estructurar las relaciones autorreferenciales, en especial la tabla de mentoria, y coordinar el orden secuencial correcto dentro del script DDL para que no fallara al ejecutarse en pgAdmin 4 debido a dependencias de claves foráneas hacia tablas que aún no se habían creado.me surgieron varias dudas al elegir ciertos tipos de datos por ejemplo la elección de claves primarias numéricas de tipo INT (SERIAL) frente a cadenas VARCHAR optimiza significativamente el uso de memoria en los índices y agiliza las operaciones de unión entre tablas. De igual forma, aprendí a valorar los campos TIMESTAMP con la propiedad DEFAULT CURRENT_TIMESTAMP, los cuales nos permiten llevar una auditoría precisa de cuándo los usuarios registran información o interactúan en la red social

JMM

Realizar esta base de datos me dejo varias enseñanzas. Primero, entendí que la base de datos no es solo un lugar para guardar información, sino que ahí mismo uno establece los límites de la aplicación. Por ejemplo, me pareció muy útil usar RESTRICT para evitar que se borre un grupo entero si el creador cancela su cuenta, o usar los CHECK para impedir situaciones ilógicas, como que un usuario se siga a sí mismo.

También resultaron varios inconvenientes. La parte más compleja fue organizar las tablas intermedias, las de muchos a muchos. Al principio pensaba ponerle a todas un ID propio, pero después de analizarlo vi que era mejor usar llaves primarias compuestas.

También tuve varias dudas durante el proceso. Por ejemplo, consideré usar BIGINT por si la red de la universidad crecía mucho, pero al investigar vi que con INT ya tenemos espacio para más de 2.000 millones de usuarios, así que no tenía sentido gastar memoria extra. Me pasó algo similar con los campos de texto: estuve a punto de usar TEXT para las biografías, pero preferí limitarlo con un VARCHAR(500) para que las descripciones no se vuelvan demasiado extensas y ocupen espacio innecesario.

Link YOUTUBE:
https://youtu.be/LzeFOj0y3NA

Link GITHUB:
https://github.com/johanmartinez927/RedSocial/tree/main/TAREA3